universität innsbruck

Variablen und Datentypen

Einführung in die Programmierung

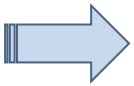
Michael Tschuggnall
Universität Innsbruck

Überblick

- Einführung
- Vorschau
- Variablen und Datentypen
- Operatoren und Ausdrücke
- Kontrollstrukturen

- Funktionen
- Arrays
- Zeiger
- Strukturen, Unions, Bitfelder
- Speicherklassen
- Dynamische Speicherverwaltung

- Präprozessor
- Modulare Programmierung
- Ein- und Ausgabe
- gcc und make



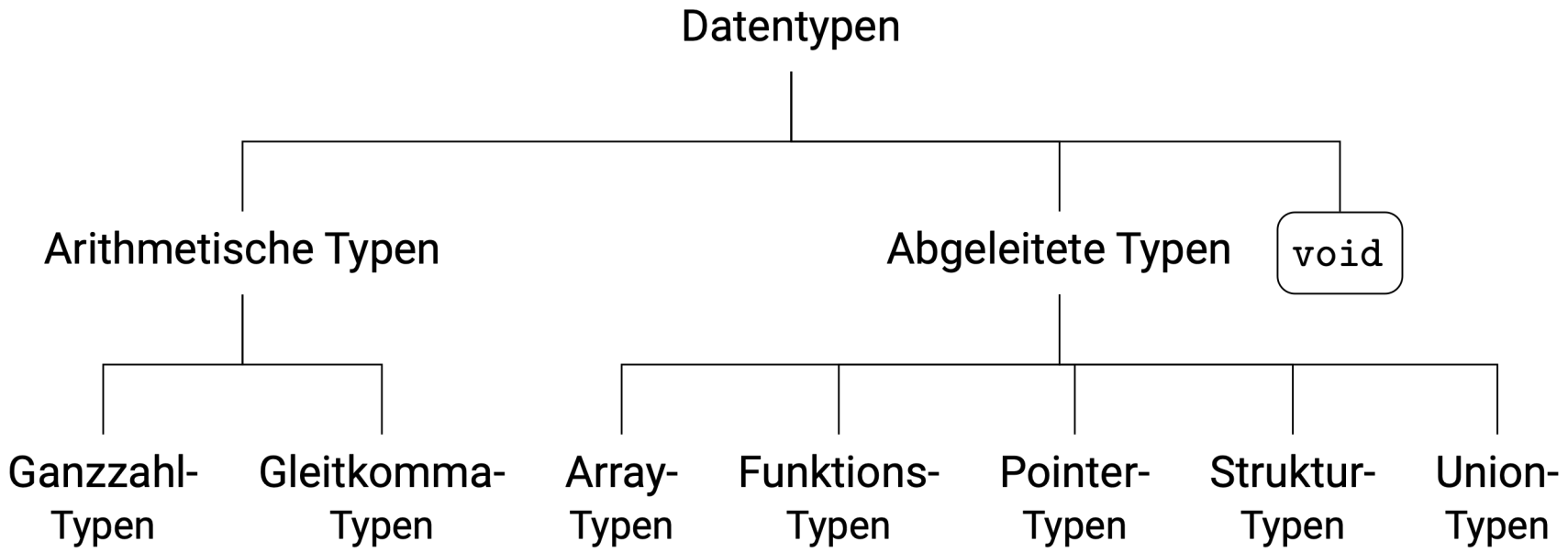
- Datentypen
- Variablen
- Literale
- Grundlegende Ein- und Ausgabe

Datentypen

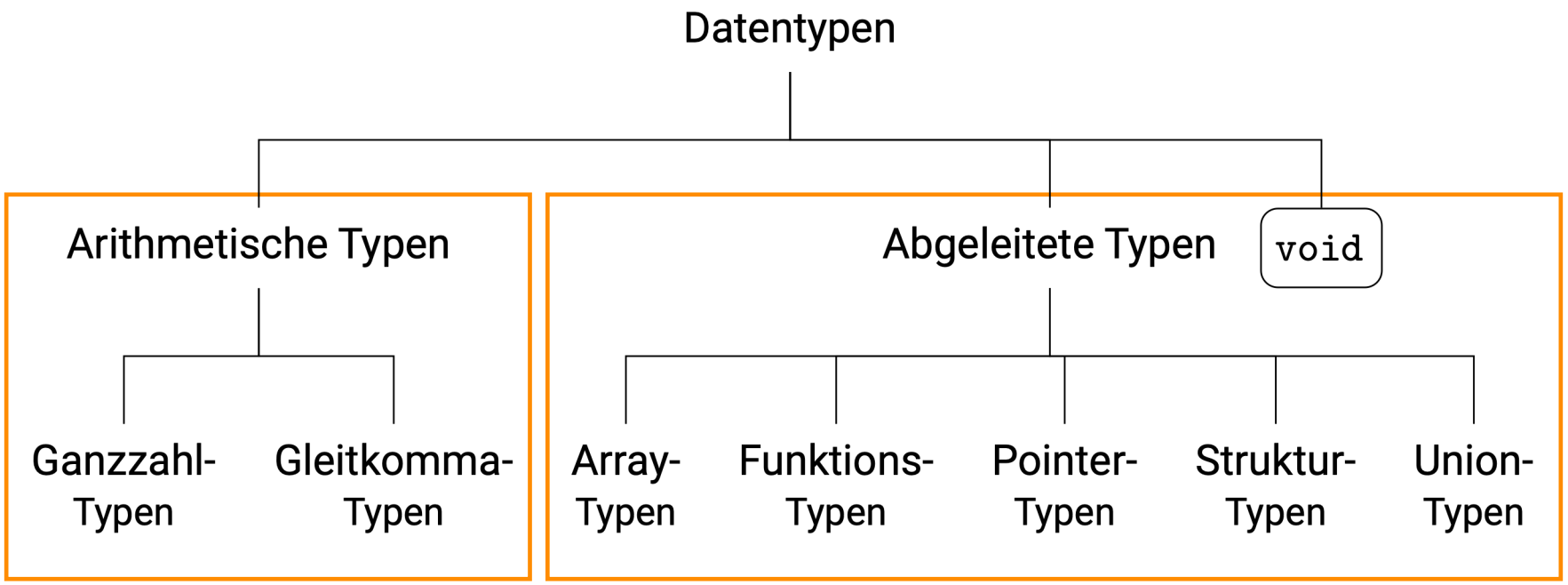
Datentypen

- Programme können verschiedene Arten von Daten speichern und verarbeiten.
- Die Interpretation der Daten wird durch den Datentyp definiert.
  - Die Binärzahl `11111111` kann abhängig vom Datentyp beispielsweise als die Dezimalzahl `255` oder `-1` interpretiert werden.
  - Daher ist eine Klassifikation dieser unterschiedlichen Daten notwendig.
- In C haben alle Daten einen genau definierten Typ.
- Der Datentyp legt fest:
  - **Wertebereich**
  - **Repräsentation im Speicher** (wie viele Bytes werden belegt)
  - **Operationen**, die auf den Datentyp angewandt werden dürfen

Typ Klassifizierung (1)



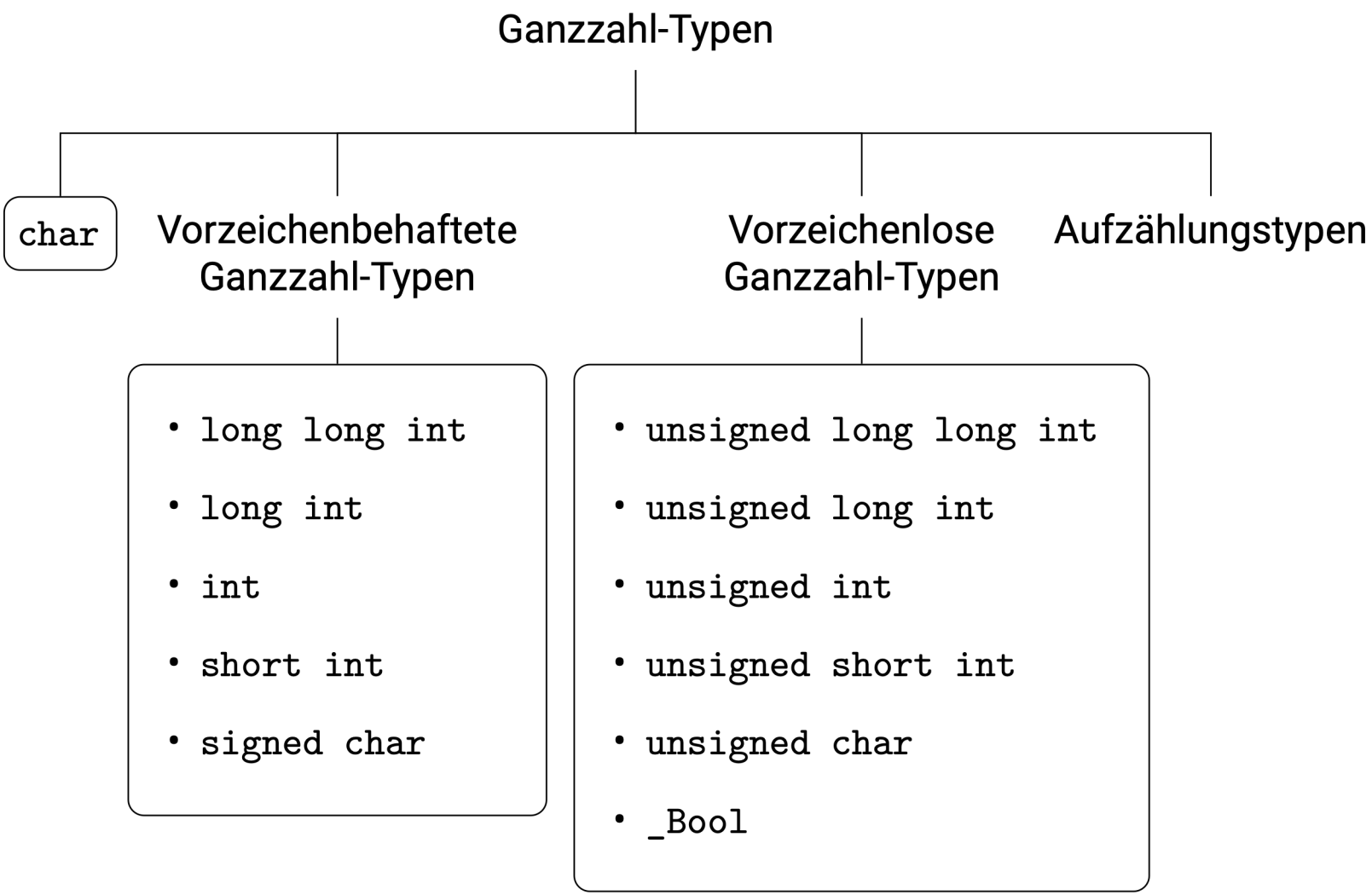
Typ Klassifizierung (2)



in dieser Vorlesung

in zukünftigen Vorlesungen

Ganzzahl-Typen (1)



Ganzzahl-Typen (2)

- Ganzzahlen werden intern in Binärdarstellung repräsentiert.
  - Vorzeichenbehaftete Ganzzahlen werden als Vorzeichen und Größenordnung, im Einerkomplement oder im Zweierkomplement dargestellt.
    - Das Zweierkomplement ist dabei die häufigste Darstellung.
    - Bei allen drei Darstellungsarten bestimmt das höchste Bit das Vorzeichen.
  - Der von einem vorzeichenbehafteten Ganzzahl-Typ darstellbare positive Wertebereich ist eine Teilmenge des entsprechenden vorzeichenlosen Ganzzahl-Typs.
    - Das Vorzeichenbit wird für die Darstellung positiver Zahlenwerte frei.
- Ordnung der Ganzzahl-Typen entsprechend ihrem Speicherbedarf bzw. dem maximal darstellbaren Wert:

$$\texttt{char} < \texttt{short} \leq \texttt{int} \leq \texttt{long} \leq \texttt{long long}$$

Datentyp char

- Der Datentyp `char` ist Teil der Ganzzahl-Typen.
- `char` ist, abhängig von der Implementierung, entweder ein Synonym für `signed char` oder für `unsigned char`.
  - `char`, `signed char` und `unsigned char` sind formal drei unterschiedliche Datentypen.
- Der von einem `char`, `unsigned char` oder `signed char` dargestellte Wert kann auf zweierlei Weise interpretiert werden:
  - Als Zeichen.
  - Als Ganzzahl.
- Mit einem `char` kann, wie mit jedem anderen Ganzzahl-Typ, auch gerechnet werden.
- Die Größe eines `char` ist immer 1 Byte.

Vorzeichenbehaftete Ganzzahl-Typen

- Das Schlüsselwort `signed` kann für vorzeichenbehaftete Ganzzahl-Typen angegeben werden.
  - Für alle vorzeichenbehafteten Ganzzahl-Typen außer `char` ist das Schlüsselwort `signed` optional.
- Synonyme für vorzeichenbehaftete Ganzzahl-Typen:

Datentyp	Synonyme
`signed char`	
`int`	`signed`, `signed int`
`short`	`short int`, `signed short`, `signed short int`
`long`	`long int`, `signed long`, `signed long int`
`long long`	`long long int`, `signed long long`, `signed long long int`

Vorzeichenlose Ganzzahl-Typen

- Das Schlüsselwort `unsigned` **muss** für vorzeichenlose Ganzzahl-Typen (außer `_Bool`) angegeben werden.
- Synonyme für vorzeichenlose Ganzzahl-Typen:

Datentyp	Synonyme
`_Bool`	
`unsigned char`	
`unsigned short`	`unsigned short int`
`unsigned int`	`unsigned`
`unsigned long`	`unsigned long int`
`unsigned long long`	`unsigned long long int`

Minimale Größe von Ganzzahlen

Datentyp	Minimale Größe in Bits	Minimum des minimalen Wertebereichs	Maximum des minimalen Wertebereichs
`char`	8	Gleich wie `signed char` oder `unsigned char`	
`signed char`	8	-127	127
`unsigned char`	8	0	255
`short`	16	-32,767	32,767
`unsigned short`	16	0	65,535
`int`	16	-32,767	32,767
`unsigned`	16	0	65,535
`long`	32	-2,147,483,647	2,147,483,647
`unsigned long`	32	0	4,294,967,295
`long long`	64	-9,223,372,036,854,775,807	9,223,372,036,854,775,807
`unsigned long long`	64	0	18,446,744,073,709,551,615

Größe der elementaren Datentypen

- Der C-Standard gibt nur minimale Größen an.
  - Die tatsächliche Größe ist abhängig vom System!
- Größen werden in den folgenden Headerdateien genau festgelegt:
  - **`limits.h`** (Ganzzahlen)
    - `INT_MAX` (Maximaler Wert eines `int`)
    - `INT_MIN` (Minimaler Wert eines `int`)
    - `UINT_MAX` (Maximaler Wert eines `unsigned int`)
    - `SHRT_MAX` (Maximaler Wert eines `short`)
    - `LONG_MAX` (Maximaler Wert eines `long`)
    - …
  - **`float.h`** (Gleitkommazahlen)
    - `DBL_MAX` (Maximaler endlicher Wert eines `double`)
    - `DBL_MIN` (Minimaler normalisierter positiver Wert eines `double`)
    - `FLT_MAX` (Maximaler endlicher Wert eines `float`)
    - …
  - **`stdint.h`** (Integer mit fester Bitbreite)

Beispiel - Größe der Ganzzahl-Typen

example_limits_int.c

```c
#include <limits.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("INT_MAX  =  %d\n", INT_MAX);
    printf("INT_MIN  = %d\n", INT_MIN);
    printf("UINT_MAX =  %u\n", UINT_MAX);
    return EXIT_SUCCESS;
}
```

Konstanten aus `limits.h`

Ausgabe auf der Standardausgabe (plattformabhängig):

```
INT_MAX  =  2147483647
INT_MIN  = -2147483648
UINT_MAX =  4294967295
```

Aufzählungstypen

- Aufzählungstypen werden auch Enumerationstypen oder kurz Enums genannt.
- Die Definition eines Enum-Typs erfolgt durch das Schlüsselwort `enum` gefolgt von einem optionalen Namen und den durch Komma getrennten Aufzählungskonstanten in geschwungenen Klammern.
- Allgemeine Form:

  `enum [Bezeichner] {Aufzaehlungskonstanten}`

- Beispiele:

```
1 enum color {RED, GREEN, BLUE};
2 enum direction {NORTH, EAST, SOUTH, WEST};
```

Gleitkomma-Typen

- Datentypen für die Darstellung von Gleitkommazahlen.
- Gleitkommazahlen haben begrenzte Genauigkeit.
  - Die einzelnen als Gleitkommazahl darstellbaren Werte stellen abhängig vom konkreten Typ eine Menge diskreter Werte dar.
  - Bei Berechnungen mit Gleitkommazahlen kann es zu Rundungsfehlern kommen.
  - Die Genauigkeit gibt an, wie viele signifikante Stellen garantiert erhalten bleiben.
- Gleitkomma-Typen:
  - `float`
    - Einfache Genauigkeit (Genauigkeit mindestens 6 Stellen)
  - `double`
    - Doppelte Genauigkeit (Genauigkeit mindestens 10 Stellen)
    - Die mit `float` darstellbaren Zahlen sind eine Teilmenge der mit `double` darstellbaren Zahlen.
  - `long double`
    - Erweiterte Genauigkeit (Genauigkeit mindestens 10 Stellen)
    - Die mit `double` darstellbaren Zahlen sind eine Teilmenge der mit `long double` darstellbaren Zahlen.

Beispiel - Genauigkeit bei Gleitkommazahlen

example_precision.c

```c
#include <float.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("FLT_DIG = %d\n", FLT_DIG);
    printf("DBL_DIG = %d\n", DBL_DIG);
    float f = 9123456e5f;
    printf(  "912345600000\n");
    printf("%f\n", f);
    printf("%e\n", f);
    return EXIT_SUCCESS;
}
```

Anzahl der Stellen von `float` bzw. `double` Werten, die in Exponentialschreibweise garantiert erhalten bleiben (plattformabhängig).

Ausgabe auf der Standardausgabe (plattformabhängig):

```
FLT_DIG = 6
DBL_DIG = 15
912345600000
912345595904.000000
9.123456e+11
```

Beispiel - Größe der Gleitkomma-Typen

example_limits_float.c

```c
#include <float.h>
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("FLT_MAX = %e\n", FLT_MAX);
    printf("DBL_MAX = %e\n", DBL_MAX);
    return EXIT_SUCCESS;
}
```

Konstanten aus `float.h`

Ausgabe auf der Standardausgabe (plattformabhängig):

```
FLT_MAX = 3.402823e+38
DBL_MAX = 1.797693e+308
```

Variablen

Variablen (1)

- Ein Programm verarbeitet Daten, die in sogenannten Variablen abgelegt werden.
- Ein Programm legt die Ergebnisse wieder in solchen Variablen ab.
- Eine Variable ist eine benannte Speicherstelle.
- Kennzeichen (für eine Variable):
  - Datentyp
  - Name
  - Wert
  - Adresse
  - Gültigkeitszeitraum
  - Sichtbarkeitsbereich

Variablen (2)

- Datentyp:
  - Daten haben einen bestimmten Typ (Datentyp).
  - Der Datentyp legt fest:
    - **Wertebereich**
    - **Repräsentation im Speicher** (z.B. wie viele Bytes werden belegt)
    - **Operationen**, die auf den Datentyp angewandt werden dürfen
- Name (Bezeichner):
  - Damit kann auf eine benannte Speicherstelle zugegriffen werden.
- Wert:
  - Der Wert einer Variable kann sich ändern.
  - Der Variable muss in bestimmten Situationen durch das Programm am Anfang explizit ein Wert zugewiesen werden (**Initialisierung**).
    - Ansonsten hat die Variable einen zufälligen Wert!
    - **Es kann nicht davon ausgegangen werden, dass die Speicherstelle mit 0 belegt ist!**

Variablen (3)

- Adresse:
  - Die Variablen liegen in den Speicherzellen des Arbeitsspeichers oder in Prozessorregistern.
  - Die Speicherzellen des Arbeitsspeichers sind durchnummeriert und diese Nummer wird als Adresse bezeichnet.
  - Eine Variable kann (und wird auch sehr oft) eine Folge zusammenhängender Speicherzellen belegen.
  - Die Adresse der Variable ist die Nummer der ersten Speicherzelle des Speicherbereichs der Variable.
- Gültigkeitszeitraum und Sichtbarkeit werden später genauer besprochen!

Variablen (4)

- Variablen müssen vor der Verwendung deklariert werden.
  - Die Deklaration legt den Datentyp und den Bezeichner fest.
  - Für die Deklaration muss mindestens der Datentyp gefolgt von einem Bezeichner angegeben werden.
- Beispiele von Deklarationen:

```c
#include <stdlib.h>

int main(void) {
    int height;
    int limit;
    char letter;
    double area;
    // ...
    return EXIT_SUCCESS;
}
```

Definition und Deklaration

- Durch eine Deklaration werden die Eigenschaften und die Art eines Bezeichners definiert.
  - Für Variablen muss der Datentyp und der Bezeichner spezifiziert werden.
- Bezeichner müssen vor der Verwendung deklariert werden.
- Definitionen sind Deklarationen, die zusätzlich dafür sorgen, dass
  - Für eine Variable zur Laufzeit Speicherplatz reserviert wird.
  - Für eine Aufzählungskonstante keine weitere Aufzählungskonstante mit demselben Bezeichner existiert.
  - Für eine Funktion der Funktionsrumpf angegeben ist (siehe Foliensatz Funktionen).
- Bei einfachen Programmen gilt: Deklaration = Definition
  - Unterschiede v.a. durch Verwendung von mehreren Dateien (siehe Foliensätze Speicherklassen und Modulare Programmierung)
  - Schlüsselwort **extern**

Bezeichner von Variablen

- Für Variablen werden Bezeichner (Namen) vergeben.
- Ein Bezeichner ist eine Folge von Buchstaben, Ziffern und dem Symbol _ (Unterstrich).
  - Der Unterstrich am Anfang sollte vermieden werden (wird nur in Bibliotheken verwendet).
  - Die Variablennamen werden stets klein geschrieben (Konvention).
  - Die meisten Systeme unterstützen auch Emojis
- Das erste Zeichen des Bezeichners darf keine Ziffer sein.
- Der Bezeichner darf kein reserviertes Schlüsselwort sein.
- C ist „case-sensitive“, d.h. in C werden Klein- und Großbuchstaben unterschieden!

Gültiger Bezeichner	Ungültiger Bezeichner
`letter`	`1letter`
`carwheel`	`car-Wheel` oder `car/Wheel`
`max_count`	`while`
`test2`	`$%`

Gute Bezeichner

- Die Namen sollten bestimmten Regeln (Konventionen) folgen!
  - Sie sollten lesbar und verständlich sein.
  - Compiler wird alle korrekten Namen akzeptieren.
  - Ein korrekter Name muss aber noch lange nicht sinnvoll sein!
  - Keine Emojis 🙈

	Negatives Beispiel	***Positives Beispiel***
Vollständige Wörter	`c` [1]	`counter`
Kleinschreibung	`COUNTER` [2]	`counter`
Neue Teile mit Unterstrich beginnen	`firstletterintext`	`first_letter_in_text`
Sinnvolle Bezeichner	`asdfg`	`counter`
Abkürzungen vermeiden	`bc`	`byte_count`
Englische Bezeichner verwenden	`erster_buchstabe`	`first_letter`

[1] Einzelne Buchstaben (wie c) sind manchmal sinnvoll bzw. werden in der Praxis verwendet (z.B. Variablen für Schleifen).
[2] Großschreibung (wie COUNTER) wird bei Konstanten verwendet.

Reservierte Schlüsselwörter

- Reservierte Schlüsselwörter (Keywords):

`auto`	`break`	`case`	`char`	`const`	`continue`
`default`	`do`	`double`	`else`	`enum`	`extern`
`float`	`for`	`goto`	`if`	`int`	`long`
`register`	`return`	`short`	`signed`	`sizeof`	`static`
`struct`	`switch`	`typedef`	`union`	`unsigned`	`void`
`volatile`	`while`				

- Zusätzliche Keywords:
  - seit C99: `restrict`, `inline`, `_Bool`, `_Complex`, `_Imaginary`
  - seit C11: `_Alignas`, `_Alignof`, `_Atomic`, `_Generic`, `_Noreturn`, `_Static_assert`, `_Thread_local`
- Schlüsselwörter können nicht als Variablennamen verwendet werden.

Literale

Literale

- Ein Literal stellt einen konstanten Wert dar.
- Der Datentyp des Literals wird durch die Art und den Wert bestimmt.
- Arten von Literalen:
  - Ganzzahlenliterale
  - Gleitkommaliterale
  - Zeichenliterale
  - String-Literale (siehe Foliensatz über Arrays)
  - Aufzählungskonstanten

Ganzzahlenliterale (1)

- Ganzzahlige Literale können im Dezimal-, Oktal- oder Hexadezimalsystem angegeben werden.

Stellenwertsystem	Präfix	Ziffernmenge
Oktalsystem	`0`	`0` - `7`
Dezimalsystem		`0` - `9`
Hexadezimalsystem	`0x`, `0X`	`0` - `9`, `a` - `f`, `A` - `F`

- Der Typ kann durch ein Suffix beeinflusst werden.
  - `unsigned:` `u` oder `U`
  - `long:` `l` oder `L`
  - `unsigned long:` `ul`, `uL`, `Ul` oder `UL`
  - `long long:` `ll` oder `LL`
  - `unsigned long long:` `ull`, `uLL`, `Ull` oder `ULL`
- Empfehlung: Großbuchstaben für Suffixe verwenden, um Verwechslungen mit `1` und `l` zu vermeiden.

Ganzzahlenliterale (2)

- Wird kein Suffix verwendet, hat das Ganzzahlenliteral den Datentyp `int` bzw. der kleinste Datentyp der Typhierarchie mit einem ausreichenden Wertebereich.
  - Für Dezimalzahlen: `int, long, long long`
  - Für Oktal- und Hexadezimalzahlen: `int, unsigned, long, unsigned long, long long, unsigned long long`
- Beispiele:
  - `123`
    - ganzzahliges Literal vom Typ `int`.
  - `123L`
    - ganzzahliges Literal vom Typ `long`.
  - `1000000000`
    - ganzzahliges Literal vom Typ `int` bzw. `long` (wenn zu groß für `int`).
  - `0XFULL`
    - Dezimalwert: `15`
    - ganzzahliges Literal vom Typ `unsigned long long`.

Gleitkommaliterale (1)

- Gleitkommaliterale können im Dezimal- oder Hexadezimalsystem angegeben werden.
- Sie bestehen aus einem Vorkommateil, einem Punkt, einem Nachkommateil, einem Exponenten und einem Suffix.
- Der Typ kann durch ein Suffix beeinflusst werden.
  - `float`: `f` oder `F`
  - `long double`: `l` oder `L`
- Wird kein Suffix angegeben, hat das Gleitkommaliteral den Datentyp `double`.
- Um ein Gleitkommaliteral von einem Ganzzahlenliteral unterscheiden zu können, muss mindestens der Dezimalpunkt oder der Exponent vorhanden sein.
  - Bei Gleitkommaliteralen im Hexadezimalsystem muss der Exponent immer angegeben werden.

Gleitkommaliterale (2)

- Beispiele
  - `10.5`
    - Gleitkommaliteral vom Typ `double`.
  - `10.5f`
    - Gleitkommaliteral vom Typ `float`.
  - `.4`
    - Gleitkommaliteral vom Typ `double`.
  - `7.5e-1f`
    - Dezimalwert: $\left(7 + \frac{5}{10}\right) \cdot 10^{-1} = 0.75$
    - Gleitkommaliteral vom Typ `float`.
  - `0xa.fp-1`
    - Dezimalwert: $\left(10 + \frac{15}{16}\right) \cdot 2^{-1} = 5.46875$
    - Gleitkommaliteral vom Typ `double`.
- Verschiedene Darstellungen sind möglich (z.B. für 0.19):
  - `0.19`
  - `.19`
  - `1.9e-1`
  - `19e-2`

Zeichenliteral (1)

- Ein Zeichenliteral umschließt mit einfachen Anführungszeichen ein Zeichen oder eine Escapesequenz.
- Escapesequenz:
  - Wird für Steuerzeichen und spezielle Zeichen verwendet.
  - Eine Escapesequenz stellt nur ein Zeichen dar.
  - Beispiele:
    - `'\n'` Zeilentrenner,
    - `'\t'` Tabulatorzeichen
    - `'\''` einfaches Hochkomma
- Zeichenliterale haben den Datentyp `int`.
  - Der Wert eines Zeichenliterals entspricht dem numerischen Wert des Zeichens im Zeichensatz der Maschine.
  - Zeichenliterale können dennoch `char`-Variablen zugewiesen werden.
- Beispiele:
  - `'A','a','%','!','5'`
  - `'\n','\t','\\','\101','\x41'`

Zeichenliteral (2)

- Der C-Standard legt nicht fest, welcher Zeichensatz verwendet wird.

Zeichensatz	Ordinalwert	Speicherbedarf	Bemerkung
ASCII	0 bis 127	7 Bit	
OEM	0 bis 255	8 Bit	kompatibel mit ASCII-Code
ANSI	0 bis 255	8 Bit	kompatibel mit ASCII-Code
ISO-Latin-1	0 bis 255	8 Bit	kompatibel mit ASCII-Code
Unicode	0 bis 1,114,112	8 Bit / 16 Bit / 32 Bit*	kompatibel mit ASCII-Code
EBCDIC	0 bis 255	8 Bit	

* Tatsächlicher Speicherbedarf hängt von der Codierung (z.B. UTF-8, UFT-16 oder UTF-32) und/oder dem kodierten Zeichen ab.

Aufgabe char-Variable

- Welche Zuweisungen führen mit unseren Flags zu einem Kompilierfehler?
  - Flags für diese VO: `-Wall -Wextra -Werror -Wpedantic -std=c18`

```
1 char c1 = '\n';
2 char c2 = '';
3 char c3 = 65;
4 char c4 = B;
5 char c5 = "B";
6 char c6 = 'C';
7 char c7 = c1;
```

Aufzählungskonstanten

- Die Aufzählungskonstanten eines Aufzählungstyps haben einen konstanten ganzzahligen Wert vom Typ `int`.
  - Die Definition des Werts erfolgt entweder implizit durch die Position oder explizit durch die Initialisierung mit einer ganzen Zahl.
  - Eine Aufzählungskonstante ohne eine Initialisierung hat den Wert 0, wenn es die erste Aufzählungskonstante ist oder den Wert der vorherigen Konstante plus Eins.
- Beispiele:

```
1  enum color {RED, GREEN, BLUE};
2  enum direction {NORTH, EAST, SOUTH = 15, WEST};
3  enum roman {I = 1, V = 5, X = 10, L = 50, C = 100, D = 500, M = 1000};
```

  - Werte der `color` und `direction` - Konstanten:
    - `RED (0), GREEN (1), BLUE (2)`
    - `NORTH (0), EAST (1),  SOUTH (15), WEST (16)`

- Bei der Zuweisung eines Werts an eine Variable eines Enum-Typs erfolgt vom Compiler keine Überprüfung, ob der zugewiesene Wert dem Wert einer Aufzählungskonstante entspricht.

Beispiel enum

example_enum.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    enum roman {I = 1, V = 5, X = 10, L = 50, C = 100, D = 500,
                M = 1000};
    enum roman my_roman = C;
    printf("%d\n", my_roman);

    enum roman my_wrong_roman = 101;
    printf("%d\n", my_wrong_roman);

    return EXIT_SUCCESS;
}
```

Nicht sinnvolle Zuweisung.
Wird vom Compiler nicht überprüft!

Ausgabe auf der Standardausgabe:

```
100
101
```

Boolsche Ausdrücke

- Vor C99 keine explizite Unterstützung
  - Programme arbeiteten mit dem Datentyp `int` und den impliziten Synonymen `0` → `false` und `1` → `true`
  - Oft eigene Typdefinitionen wie `typedef int bool;` (wird später besprochen)
- Seit C99 zwei Unterstützungen
  1. neuer vollwertiger Datentyp `_Bool`
     - Kann nur die Werte 0 oder 1 annehmen
     - Keine Schlüsselwörter für `true` und `false`
     - Speichergröße nicht im Standard vorgeschrieben, meist aber 1 oder 2 Byte

     ```
     _Bool a = 1;
     ```

  2. neuer Header `<stdbool.h>`
     - Ermöglicht die Verwendung von `bool`, `true` und `false`
     - Gelöst über Makros (wird später besprochen), wo folgende Ersetzungen stattfinden:
       - `bool` → `_Bool`, `true` → `1`, `false` → `0`
     - Implizit wird also auch hier `_Bool` verwendet

     ```
     bool b = true;
     ```

- Es gibt keine expliziten Literale für _Bool
  - z.B. existiert kein `1b` oder `0B` wie z.B. bei `long` (`1L`)
  - Damit immer implizite Typumwandlung (siehe nächster Foliensatz)

Grundlegende Ein- und Ausgabe

Ausgabe mit printf (1)

- `printf` gehört nicht zur Sprache C selbst, sondern ist eine Funktion der C Standard Library.
- Allgemeine Form von `printf`:
  - **`printf(Formatstring, Parameterliste)`**
- Formatstring:
  - Besteht aus gewöhnlichem Text, in den Platzhalter zur Ausgabe von Variableninhalten eingefügt werden können.
  - Die Platzhalter beginnen immer mit einem Prozentzeichen (%).
  - Platzhalter können auch Angaben über das Ausgabeformat enthalten.
- Parameterliste:
  - Die Parameterliste hängt vom Formatstring ab und fällt ggf. ganz weg.
  - Die Parameterliste kann Konstanten, Variablen oder Ausdrücke enthalten.
- Rückgabewert:
  - Ganzzahl, welche der Anzahl der ausgegebenen Zeichen entspricht.

Ausgabe mit printf (2)

- Platzhalter (Formatspecifier) im Formatstring haben die folgende Struktur:

  `%[flags][width][.precision][modifier]converter`

  - Angaben in Klammern sind optional.

  `[flags]`

  - Zum Steuern des Verhaltens der Umwandlung.

  `[width]`

  - Entspricht der Mindestanzahl der ausgegebenen Zeichen.

  `[.precision]`

  - Entspricht der Präzision. Hat abhängig vom Datentyp unterschiedliche Bedeutung.

  `[modifier]`

  - Spezifiziert in Kombination mit dem Converter den erwarteten Datentyp des Parameters.

  `converter`

  - Spezifiziert die Art der Umwandlung.

printf – converter

Converter	Bedeutung	Datentyp des Parameters
d, i	Ausgabe einer vorzeichenbehafteten Ganzzahl als Dezimalzahl.	`int`
u	Ausgabe einer vorzeichenlosen Ganzzahl als Dezimalzahl.	`unsigned int`
o	Ausgabe einer vorzeichenlosen Ganzzahl als Oktalzahl.	`unsigned int`
x, X	Ausgabe einer vorzeichenlosen Ganzzahl als Hexadezimalzahl in Klein- bzw. Großschreibung.	`unsigned int`
c	Ausgabe eines einzelnen Zeichens.	`int`
s	Ausgabe einer Zeichenkette (wird noch ausführlich besprochen).	`char*`
f	Ausgabe einer Gleitkommazahl als Dezimalzahl.	`double`
e, E	Ausgabe einer Gleitkommazahl als Dezimalzahl in Exponentialdarstellung mit kleingeschriebenem „e“ bzw. großgeschriebenem „E“.	`double`
g, G	Ausgabe einer Gleitkommazahl als Dezimalzahl oder Dezimalzahl in Exponentialdarstellung in Abhängigkeit vom Wert.	`double`
a, A (C 99)	Ausgabe einer Gleitkommazahl als Hexadezimalzahl in Exponentialdarstellung.	`double`
p	Ausgabe als Adresse (Ausgabe ist implementierungsabhängig).	`void*`
n	Speichert in einer als Argument angegebenen Variable des Typs `int` die Anzahl der bisher ausgegebenen Zeichen.	`int*`
%	Ausgabe des %-Zeichens.	

Beispiel printf – converter

example_printf_converter.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("a) |%d| |%d|\n", 1, -1);
    printf("b) |%u| |%u|\n", 2, -2);
    printf("c) |%x|\n", 10);
    printf("d) |%d| |%c| |%c|\n", 'A', 65, 'A');
    printf("e) |%f| |%e|\n", 30.5, 30.5);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe (plattformabhängig):

```
a) |1| |-1|
b) |2| |4294967294|
c) |a|
d) |65| |A| |A|
e) |30.500000| |3.050000e+01|
```

printf – modifier

Modifier	Datentyp des Parameters (abhängig vom verwendeten Converter)	Beispiel
h	signed short unsigned short	hd hu
hh	signed char unsigned char	hhd hhu
j	intmax_t uintmax_t	jd ju
l	signed long unsigned long	ld lu
ll	signed long long unsigned long long	lld llu
t	ptrdiff_t unsigned Version von ptrdiff_t	td tu
z	signed Version von size_t size_t	zd zu
L	long double	Lf

Converter für Ganzzahlen:

- i, d (Vorzeichenbehaftet)
- o, u, x, X (Vorzeichenlos)

Converter für Gleitkommazahlen:

- a, A, e, E, f, F, g, G

Beispiel printf – modifier

example_printf_modifier.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    signed char cs1 = 1;
    signed char cs2 = -1;
    printf("a) |%hhd| |%hhd|\n", cs1, cs2);
    printf("b) |%hhu| |%hhu|\n", cs1, cs2);
    printf("c) |%ld| |%lu|\n", 2L, -2L);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe (plattformabhängig):

```
a) |1| |-1|
b) |1| |255|
c) |2| |18446744073709551614|
```

printf – flags

- Die Verwendung von einem oder mehreren der nachfolgenden Flags verändert das Verhalten der Umwandlung.
  - `-` (linksbündige Ausgabe)
  - `+` (positive Zahl mit Vorzeichen ausgeben)
  - `0` (mit führenden Nullen auffüllen)
  - `#` (alternative Form) siehe Dokumentation: `man 3 printf`
  - Leerzeichen (wird vorangestellt, wenn das erste Zeichen kein Vorzeichen ist)

example_printf_flags.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("a) |%d|\n", 1);
    printf("a) |%d|\n", -1);
    printf("-------\n");
    printf("b) |% d|\n", 2);
    printf("b) |% d|\n", -2);
    printf("-------\n");
    printf("c) |%+d|\n", 3);
    printf("c) |%+d|\n", -3);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a) |1|
a) |-1|
-------
b) | 2|
b) |-2|
-------
c) |+3|
c) |-3|
```

printf – width

- Durch die Angabe von `width` kann die minimale Ausgabebreite angegeben werden.
  - Die Breite kann direkt als Ganzzahl oder mittels * und entsprechendem Parameter in der Parameterliste angegeben werden.
  - Die Ausgabe wird mit Leerzeichen aufgefüllt, um die minimale Ausgabebreite zu erreichen (space padding).
  - Ist die Ausgabe länger als die minimale Ausgabebreite, so wird der Wert für `width` ignoriert.

example_printf_width.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("a) |%3d|\n", 1);
    printf("b) |%3d|\n", -1);
    printf("c) |%-3d|\n", 1);
    printf("d) |%03d|\n", 1);
    printf("e) |%03d|\n", 1234);
    printf("f) |%*d|\n", 3, 1);
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe:

```
a) |  1|
b) | -1|
c) |1  |
d) |001|
e) |1234|
f) |  1|
```

printf – .precision

- Abhängig vom Datentyp hat `.precision` eine andere Bedeutung:
  - Converter `d`, `i`, `o`, `u`, `x`, und `X`:
    - Mindestanzahl der ausgegebenen Stellen.
    - Bei Bedarf wird vorne mit Nullen aufgefüllt.
    - Sind sowohl Präzision als auch der auszugebende Wert 0, wird nichts ausgegeben.
  - Converter `a`, `A`, `e`, `E`, `f`, und `F`:
    - Anzahl der Nachkommastellen.
    - Bei Bedarf werden Nullen angehängt.
  - Converter `g` und `G`:
    - Maximale Anzahl signifikanter Nachkommastellen.
  - Converter `s`:
    - Maximale Anzahl der ausgegebenen Bytes.
- Die Präzision kann nach dem `.` entweder direkt als Ganzzahl oder mittels * und entsprechendem Parameter in der Parameterliste angegeben werden.

Beispiel printf – .precision

example_printf_precision.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("a) |%.0d|\n", 1);
    printf("a) |%.0d|\n", 0);
    printf("-------\n");
    printf("b) |%2.2f|\n", 0.899);
    printf("b) |%5.2f|\n", 0.899);
    printf("b) |%.1f|\n", 20.11);
    printf("-------\n");
    printf("c) |%s|\n", "Hello World!");
    printf("c) |%.5s|\n", "Hello World!");
    return EXIT_SUCCESS;
}
```

Ausgabe auf der Standardausgabe (plattformabhängig):

```
a) |1|
a) ||
-------
b) |0.90|
b) | 0.90|
b) |20.1|
-------
c) |Hello World!|
c) |Hello|
```

scanf

- Die Funktion `scanf` liest zeichenweise eine Folge von Eingabefeldern ein.
  - Für jeden Formatspecifier muss eine Adresse auf eine Variable des entsprechenden Datentyps vorhanden sein.
  - Die Adresse einer Variable `x` kann durch `&x` ermittelt werden.
- Die Platzhalter sind ähnlich zu `printf`.
  - `scanf` liest aber ein!
  - Manual-Seite studieren (`man scanf`)!
    - Für das Einlesen eines double-Werts muss beispielsweise `%lf` verwendet werden.
- Rückgabewert:
  - Ganzzahl, welche der Anzahl der Werte, die erfolgreich zugewiesen werden konnten, entspricht.

Beispiel scanf

example_scanf.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int i;
    printf("Bitte geben Sie eine Zahl ein: ");
    if (scanf("%d", &i) != 1) {
        printf("Fehler beim Einlesen des Werts für i!\n");
        return EXIT_FAILURE;
    }
    printf("Die Zahl, die Sie eingegeben haben, war %d\n", i);
    return EXIT_SUCCESS;
}
```

Aufgabe scanf

- Das nachfolgende Codefragment soll zwei Ganzzahlen vom Typ `int` nacheinander einlesen. Identifizieren Sie alle Fehler und korrigieren Sie diese!

exercise_scanf.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    int var1 = 0;
    int var2 = 0;
    printf("Zahl 1 eingeben:");
    scanf("%d", var1);
    printf("Zahl 2 eingeben:");
    scanf("%s", var2);
    printf("%d + %d = %d\n", var1 + var2);
    return EXIT_SUCCESS;
}
```